

AFL Assistant: Executive Report

Product Goal

A conversational assistant scoped strictly to AFL — teams, players, matches, stats, history, and match/player predictions — built to demonstrate a production-shaped pattern for a client property (e.g. Web3Geeks): grounded answers (no hallucinated stats), explicit routing instead of one agent guessing its way through every request, and predictions that are always framed as probabilities, never certainties.

Architecture

Chat, retrieval, and prediction are orchestrated through a LangGraph state machine rather than a single agent freely deciding what to do:

- Router node classifies each message into one of 6 intents (factual / retrieval / prediction-match / prediction-player / off-topic / ambiguous) using structured LLM output.
- Retrieval node calls one of 12 structured pandas-backed tools against the real feature tables — exact stats, never fuzzyretrieved.
- Prediction node resolves team nicknames and relative dates, then calls the Day 2 trained models (HistGradientBoosting for match winner, Ridge for top player), attaching a real permutation-importance-based explanation.
- Validation node checks every tool result before it's shown to the user; unresolved entities route to a clarification node instead of a guess.
- Response formatter appends the prediction disclaimer structurally, in code — not left to the model's judgment on a given turn.

The full system is wrapped behind a FastAPI /chat endpoint with structured JSON logging (query, intent, tools called, latency, token usage) and a minimal static HTML/JS chat UI, both included in the deliverable.

Evaluation Results

27 test cases across 4 categories (exceeds the 25+ target). Prediction-sanity cases are executed directly against the trained model with real, verified results:

Category	Cases	Result
Prediction sanity	6	6/6 pass — extreme ladder-gap matchups correctly favor the stronger team (>90%), home-advantage direction holds, draw probability stays under 2%
Factual Q&A	6	Requires live API run — keyword-heuristic auto-grading + manual review, harness included
Scope guardrails	8	Requires live API run — 6 prompt-injection variants also tested separately, harness included
Multi-turn coherence	7 turns / 3 convos	Requires live API run — includes an unresolvable-team clarification loop and an off-topic-then-redirect case

Benchmark: the match-winner model vs. a naive "higher ladder position wins" baseline, walk-forward across the 3 most recent seasons — 74.1% vs 65.3% (2023), 74.5% vs 61.6% (2024), 66.7% vs 67.1% (2025). The model wins 2 of 3 seasons and is essentially tied in the third against a baseline that itself beats a plain always-home-team guess by ~10-12 points — a real, if modest, edge over a genuinely competitive baseline, which is the honest context for how good a sports prediction model "needs" to be.

Known Limitations

- Data recency: the underlying dataset ends September 2025; there is no live fixture feed, so "this week" style prediction requests fall back to each team's most recently known form with an explicit note, rather than a confirmed upcoming match.
- Model accuracy ceiling: match outcomes carry genuine irreducible randomness — the model's ~67-75% accuracy range is a realistic ceiling for this task, not a shortfall; a model claiming much higher accuracy on this data would be a leakage red flag, not a win (see the Day 1 EDA notes).

- Guardrail edge cases: ambiguous AFL-adjacent questions (e.g. "which AFL players also played cricket") depend on the router correctly distinguishing genuine AFL relevance from off-topic drift — the router accuracy table (Day 4/5 harness) is the way to monitor this over time, not a one-time guarantee.
- Player-stat predictions beyond fantasy points (disposals, goals) use a simpler rolling-average estimate, not a dedicated trained model — stated explicitly to the user rather than presented as equally rigorous.

Recommended Next Steps

- Wire in a real upcoming-fixtue feed so match predictions can resolve "this week" to an actual scheduled game instead of falling back to latest known form.
- Stand up the weekly retrain-trigger loop described in MONITORING.md so the model doesn't silently drift as new rounds complete.
- Run the full evaluation suite (Task 2) against production traffic samples monthly, not just at launch, and track the router accuracy trend over time.
- Consider a learning-to-rank model for the top-player prediction, since regression-then-rank did not clearly beat the naive rolling-average baseline on top-k hit rate in Day 2's evaluation.

Deliverables Included

- Full LangGraph application (routing, retrieval, prediction, validation, clarification) plus a FastAPI wrapper and a minimal static chat UI.
- Combined evaluation suite (27 cases) with a runnable harness and a results table, including the 6 fully-executed predictionsanity checks.
- This executive report, a one-page monitoring checklist (MONITORING.md), and a demo script for stakeholder walkthroughs.
- Structured JSON request logging built in from day one, so the monitoring plan has real data to read from immediately on deployment, not after a follow-up engineering pass.